

Lecture 4: Data Cleaning & Preprocessing

Week 2: Computational Linguistics

LLM Course

Learning Objectives [?]

By the end of this lecture, you will be able to:

1. Understand why data cleaning is critical for NLP tasks
2. Apply common preprocessing techniques to raw text
3. Use web scraping to collect text data
4. Implement lemmatization and stemming
5. Make informed decisions about preprocessing strategies

Key theme: Garbage in, garbage out! [?] → [?]

Why Data Cleaning Matters ?

Real-world text is messy!

Common problems:

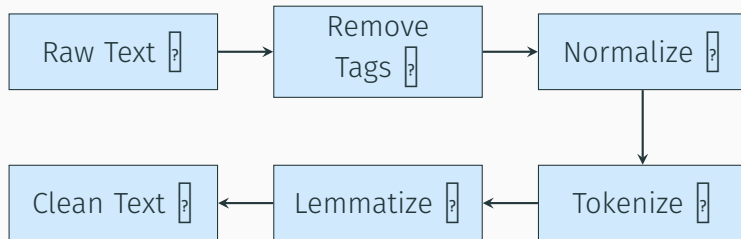
- HTML/XML tags: `<p>text</p>`
- Special characters: ` `; `&`;
- Inconsistent formatting
- Extra whitespace
- Mixed encodings (UTF-8, ASCII...)
- Typos and misspellings
- Emoji and Unicode ? ?

Consequences of dirty data:

- Models learn noise, not signal
- Reduced accuracy
- Inconsistent predictions
- Poor generalization
- Wasted computation

"Quality of input = quality of output"

The Data Cleaning Pipeline [?]



Note: Pipeline varies by task! Not all steps are always needed.

Task-Specific Preprocessing [?]

Different tasks need different preprocessing:

Task	Keep	Remove/Modify
Sentiment Analysis	Punctuation (!?) Emoji	HTML tags, URLs Extra whitespace
Named Entity Recognition	Case (names) Punctuation	HTML tags Special chars
Text Generation	Everything! (preserve style)	Minimal cleaning (just corrupted text)
Topic Modeling	Content words	Stop words, punctuation Case (lowercase)

Key principle: Preserve information relevant to your task! [?]

Why web scraping?

- Web = massive source of text data
- News articles, reviews, social media, forums...
- Fresh, diverse, domain-specific content

Popular tools:

- **Beautiful Soup:** Parse HTML/XML, extract text
- **Requests:** Fetch web pages
- **Scrapy:** Full-featured web scraping framework
- **Selenium:** For JavaScript-heavy sites

Important considerations:

- Check `robots.txt` and terms of service
- Respect rate limits (don't overwhelm servers)

Web Scraping with BeautifulSoup

Basic example:

```
from bs4 import BeautifulSoup
import requests

# Fetch webpage
url = "https://example.com/article"
response = requests.get(url)
html_content = response.content

# Parse HTML
soup = BeautifulSoup(html_content, 'html.parser')

# Extract text from specific elements
title = soup.find('h1').get_text()
article = soup.find('article').get_text()
```

```
from bs4 import BeautifulSoup

html = """<div class="article">
    <h2>Breaking News</h2>
    <p class="content">First paragraph.</p>
    <p class="content">Second paragraph.</p>
    <div class="ads">Advertisement</div>
</div>"""

soup = BeautifulSoup(html, 'html.parser')

# Find all paragraphs with class="content"
paragraphs = soup.find_all('p', class_='content')
text = ' '.join([p.get_text() for p in paragraphs])

# Remove unwanted sections
```

Handling Text Encodings ❓

Common encoding issues:

- UTF-8 vs. ASCII vs. Latin-1
- Special characters: é, ñ, ❓❓
- Emoji: ❓ (requires Unicode support)

Best practices:

- Always use UTF-8 when possible
- Detect encoding: **chardet** library
- Normalize Unicode: NFKC vs. NFD forms

Example problem:

Input: `caf\xe9` (Latin-1)

Decoded: `café` (UTF-8)

Common Preprocessing Steps ?

1. HTML/XML Tag Removal

- Strip markup: `<p>Hello</p>` → `Hello`
- Tools: BeautifulSoup, regex, html2text

2. Whitespace Normalization

- Remove extra spaces, tabs, newlines
- `"Hello world\n"` → `"Hello world"`

3. Punctuation Handling

- Keep for sentiment: `"Great!"` vs. `"Great"`
- Remove for topic modeling

4. Case Normalization

- Lowercase: `"Apple"` and `"apple"` → same token

Preprocessing Example ?

```
import re

def preprocess_text(text):
    # Remove URLs
    text = re.sub(r'http\S+|www.\S+', '', text)

    # Remove email addresses
    text = re.sub(r'\S+@\S+', '', text)

    # Remove HTML tags
    text = re.sub(r'<.*?>', '', text)

    # Remove extra whitespace
    text = ' '.join(text.split())

    # Lowercase (optional)
```

Reducing Words to Base Forms [?]

Goal: Reduce inflected/derived words to root form

Stemming [?]

- Rule-based, crude chopping
- Fast, simple
- May produce non-words
- Porter Stemmer (1980)
- Example: "running" → "run"

Lemmatization [?]

- Uses vocabulary + morphology
- Slower, more accurate
- Produces real words
- Requires POS context
- Example: "better" → "good"

When to use which?

- Stemming: Speed matters, approximate matching OK
- Lemmatization: Need interpretable, real words

Stemming vs. Lemmatization Examples

Original Word	Stemming	Lemmatization
running	run	run
ran	ran	run
runs	run	run
runner	runner	runner
better	better	good
best	best	good
mice	mice	mouse
geese	gees	goose
organization	organ	organization
organizing	organ	organize
meeting	meet	meeting (or meet)

Stemming with NLTK ?

```
from nltk.stem import PorterStemmer, SnowballStemmer

porter = PorterStemmer()
snowball = SnowballStemmer('english')

words = ['running', 'runs', 'runner', 'ran', 'easily', 'fairly']

print("Word           Porter           Snowball")
print("-" * 40)
for word in words:
    p_stem = porter.stem(word)
    s_stem = snowball.stem(word)
    print(f"{word:12} {p_stem:12} {s_stem}")
```

Output:

Word Porter Snowball

Lemmatization with spaCy

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = "The cats are running faster than dogs ran yesterday"
doc = nlp(text)

print(f"{'Word':<12} {'Lemma':<12} {'POS':<12}")
print("-" * 36)
for token in doc:
    print(f"{token.text:<12} {token.lemma_:<12} {token.pos_:<12}")
```

Output:

<i># Word</i>	<i>Lemma</i>	<i>POS</i>
<i># -----</i>		
<i># The</i>	<i>the</i>	<i>DET</i>

spaCy vs. NLTK for Lemmatization ?

Feature	spaCy	NLTK
Speed	? Fast (Cython)	? Slower (Python)
Accuracy	? High	? Good
Setup	Easy	Requires data download
POS tagging	Built-in	Separate step
Dependencies	Included	Manual WordNet
Use case	Production	Research/Teaching

Recommendation: Use spaCy for most tasks!

It's faster, more accurate, and easier to use in production.

What are stop words?

- Common words with little semantic value
- Examples: "the", "a", "is", "in", "and", "or"
- Language-specific

When to remove?

- ? Topic modeling, text classification
- ? Search engines, information retrieval
- ? Sentiment analysis ("not good" vs. "good")
- ? Machine translation
- ? Text generation

Warning: Modern neural models often don't need stop word removal!

They can learn to ignore them or use them for grammar

```
from nltk.corpus import stopwords
import spacy

# NLTK approach
stop_words_nltk = set(stopwords.words('english'))
text = "This is an example showing stop word removal"
tokens = text.lower().split()
filtered_nltk = [w for w in tokens if w not in stop_words_nltk]

print("NLTK:", filtered_nltk)
# Output: ['example', 'showing', 'stop', 'word', 'removal']

# spaCy approach
nlp = spacy.load("en_core_web_sm")
doc = nlp(text)
filtered_spacy = [token.text for token in doc
```

Case Study: Preprocessing for Sentiment Analysis

Scenario: Analyze customer reviews from Amazon

Raw review example:

"I LOVE this product!!! Best purchase ever! See details at <http://example.com>"

Preprocessing decisions:

- Keep: Punctuation (!), emoji (👍), capitalization (emphasis)
- Remove: URLs, HTML tags
- Maybe: Extra repeated punctuation (!!!)

Result:

"I LOVE this product! Best purchase ever!"

Questions to consider:

1. **Information loss:** What do we lose when we lowercase everything?
 - Think: "US" (United States) vs. "us" (pronoun)
2. **Task dependency:** Why does preprocessing differ by task?
 - Hint: What signals matter for sentiment vs. topic modeling?
3. **Modern models:** Do transformer models still need heavy preprocessing?
 - Consider: BERT handles subwords, capitalization, punctuation...
4. **Bias introduction:** Can preprocessing introduce bias?
 - Example: Removing slang might remove cultural markers

Best practices:

1. **Inspect your data first!** ?
 - Look at samples before deciding on preprocessing
2. **Keep raw data separate** ?
 - Never overwrite originals—you might need them!
3. **Document your pipeline** ?
 - Track what preprocessing you applied and why
4. **Experiment!** ?
 - Try different approaches, measure impact on task
5. **Validate incrementally** ?
 - Check results after each preprocessing step
6. **Consider automation** ?
 - Use libraries: spaCy, NLTK, HuggingFace tokenizers

Complete Preprocessing Pipeline Example ?

```
import spacy
import re

class TextPreprocessor:
    def __init__(self, remove_stopwords=False):
        self.nlp = spacy.load("en_core_web_sm")
        self.remove_stopwords = remove_stopwords

    def clean(self, text):
        # Remove URLs
        text = re.sub(r'http\S+', '', text)
        # Remove HTML tags
        text = re.sub(r'<.*?>', '', text)
        # Normalize whitespace
        text = ' '.join(text.split())
```

Preprocessing Checklist [?]

Before starting your project, ask:

- ☐ What is my task? (classification, generation, extraction...)
- ☐ What signals are important? (sentiment, topics, entities...)
- ☐ Should I lowercase? (preserve case for names?)
- ☐ How to handle punctuation? (keep for emotion?)
- ☐ Do I need lemmatization? (or will model handle it?)
- ☐ Should I remove stop words? (or keep for grammar?)
- ☐ How to handle special characters? (emoji, numbers, symbols)
- ☐ What about rare/unknown words? (keep, remove, replace?)
- ☐ Have I validated on sample data? (inspect before/after!)

Remember: There's no one-size-fits-all solution! [?]

Web Scraping:

- BeautifulSoup:
`https://www.crummy.com/software/BeautifulSoup/`
- Scrapy: `https://scrapy.org/`

Text Processing:

- spaCy: `https://spacy.io/`
- NLTK: `https://www.nltk.org/`
- TextBlob: `https://textblob.readthedocs.io/`

Encoding:

- chardet: Character encoding detection
- ftfy: Fixes broken Unicode

Classic papers:

- Porter, M. F. (1980). An algorithm for suffix stripping. *Program*, 14(3), 130-137.
 - The original Porter Stemmer algorithm
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
 - Chapter 2: Text preprocessing fundamentals

Modern resources:

- spaCy documentation: Industrial-strength NLP
- HuggingFace NLP Course: Modern preprocessing with transformers

Ethical considerations:

- Liang et al. (2020). "Towards Debiasing Sentence Representations"
- Consider how preprocessing choices affect fairness

Try this yourself:

1. Choose a website (news, blog, Reddit...)
2. Scrape 10-20 articles/posts
3. Apply different preprocessing pipelines:
 - Minimal: Just remove HTML
 - Moderate: + normalize whitespace, lowercase
 - Heavy: + lemmatize, remove stop words
4. Compare the results:
 - How does vocabulary size change?
 - What information is lost/preserved?
 - Which would work best for sentiment analysis? Topic modeling?

Bonus: Share interesting findings with classmates! ?

Key Takeaways [?]

1. **Preprocessing is crucial** but task-dependent
 - No universal pipeline—adapt to your needs!
2. **Balance cleaning vs. information loss**
 - More preprocessing $\neq$ always better
3. **Stemming vs. Lemmatization**
 - Stemming: fast, approximate
 - Lemmatization: slow, accurate
4. **Modern models are robust**
 - Transformers can handle messy text better than older models
5. **Always validate**
 - Inspect data before and after preprocessing

Next lecture: Tokenization deep dive! [?]

Questions? 

Next: Lecture 5 - Tokenization